

Sideflash

One embedded address for Ark and Lightning

Whitepaper v0.2 | 4 October 2026 | Experimental, not audited

Abstract

Sideflash combines a native Ark destination and a reusable BOLT12 offer in one authenticated address. A compatible wallet selects the native Ark route when the sender and recipient use the same Ark service provider (ASP). Across ASPs, the intended route uses Lightning with separate safeguards for delivery into the recipient's Ark balance. A Lightning-only client can extract the embedded offer without an Ark wallet or an address lookup service.

The address binds both destinations to the recipient's receiving key and the destination ASP's full public key. It includes network context, revision, validity and two signatures. These authenticate the binding; they do not prove liquidity, fresh registration or successful Ark delivery.

What changes in v0.2

This revision specifies the compact, fully embedded **wire profile v1**. A fixed CBOR array replaces the original map. A built-in network-profile code replaces two explicit 32-byte hashes without removing their verification semantics. The complete offer, native destination and both signatures remain embedded. There is no resolver or directory lookup.

Frozen example	Legacy v0	Compact v1
Mainnet	911 characters	785 characters
Regtest	967 characters	841 characters

Both examples save 126 characters. The mainnet example is approximately 13.8% shorter. Length depends on offer size and integer widths; these are measurements, not a fixed address length.

Version distinction: v0.2 is the whitepaper revision. Version 1 is the encoded address format. The 1 in sf11 is only the Bech32 separator.

1 Compact address anatomy

An address has four outer parts. Its fields are encoded as one binary payload; field boundaries are not independent readable words in the text.

Prefix	Separator	Encoded payload	Checksum
sfl	1	5-bit Bech32 symbols	6 Bech32m symbols

For the 785-character mainnet fixture, positions 1-3 are the prefix, position 4 is the separator, positions 5-779 carry the payload, and positions 780-785 carry the checksum. These ranges apply only to that fixture.

Canonical CBOR array

Index	Type	Content
0	uint64	Wire version: 1
1	uint64	Required envelope features: 0
2	uint64	Built-in network profile: 0 or 1
3	33-byte string	Full compressed ASP public key
4	byte string	Binary native Ark destination
5	byte string	Complete, exact BOLT12 offer TLV bytes
6	uint64	Binding revision
7	uint64	Inclusive valid-from time, Unix seconds
8	uint64	Exclusive expiry time, Unix seconds
9	64-byte string	Recipient BIP340 authorization
10	64-byte string	ASP BIP340 acknowledgment

The array contains exactly 11 elements. Use definite lengths and shortest integer/length encodings. Reject tags, booleans in integer fields, negative or overflowing integers, extra elements and trailing bytes. Unknown versions, network profiles, required features and recipient policies fail explicitly.

The outer limits remain **633 decoded bytes and 1023 ASCII characters**. Reject oversized input before nested parsing. Accept uniformly lowercase or uppercase text; reject mixed case and invalid padding. Uppercase QR encoding can reduce QR size. Never truncate an offer or replace it with a lookup reference.

2 Network identity and signatures

Built-in network profiles

Version 1 defines local development assignments: code **0** identifies bitcoin (XBT) mainnet; code **1** identifies isolated bitcoin (XBT) regtest. Implementations ship the same immutable table. Each code expands locally to a genesis hash and fork discriminator. No network request is necessary. These are not globally assigned chain identifiers.

Genesis hashes below use Lightning chain-hash byte order:

0: 6fe28c0ab6f1b372c1a6a246ae63f74f931e8365e15a089c68d6190000000000

1: 06226e46111a0b59caaf126043eb5bbf28c34f3a5e332a1fc7b2b73cf188910f

Both use the existing discriminator: SHA256 of the ASCII string below, with no terminating NUL. Its spelling is a frozen protocol identifier.

Sideflash/XBT/blake2b-unified-sighash/v0

A reader compares the expanded profile with its configured chain, the native network and the offer chain. The offer must require identity bit 512; unsupported mandatory offer features are rejected. Signet and other networks are not assigned. A shared genesis or address prefix is insufficient. Regtest does not identify a particular laboratory; full ASP identity pinning is still required.

Two authorizations, retained

Let $A(n)$ be canonical CBOR encoding of the first n array elements, including an array header of length n . The signature digests are:

```
recipient_digest = SHA256(
  "Sideflash/recipient/v1" || 0x00 || A(9))
service_digest = SHA256(
  "Sideflash/server/v1" || 0x00 || A(10))
```

Element 9 signs the recipient digest under the native pubkey-policy receiving authority. Element 10 signs the service digest under the x-only form of element 3. The ASP acknowledgment includes the recipient signature. Neither party needs the other party's secret key.

Authenticate the recipient channel and pin the full ASP identity from trusted context. A short native fingerprint is not an identity anchor. Valid signatures do not prevent wholesale substitution with an attacker's separately valid address. The binding signatures do not replace BOLT12 invoice signatures or transaction signing rules.

3 Savings, parsing and migration

Fixture	v0 payload	v1 payload	Text saving
Mainnet	563 bytes	484 bytes	126 characters
Regtest	598 bytes	519 bytes	126 characters

For these fixtures, two 32-byte CBOR byte strings become one small profile integer: 67 bytes saved. Removing the original map's 12 integer keys saves 12 more bytes. Total: **79 payload bytes**. No signature is removed. No offer is truncated. The Ark destination and offer bytes are identical to those in the legacy fixtures.

Verify before use

First check text bounds, prefix, checksum and padding. Then parse the exact versioned structure, reject noncanonical encodings, and enforce nested bounds. Validate native policy lengths before allocation. The initial policy is a pubkey destination; delegated or scripted authorities are not supported.

Derive the recipient verification key from the native policy, check its ASP fingerprint against the full pinned key, verify network and validity, then verify both signatures. Validate the complete offer under BOLT12, including chain, expiry and mandatory features. Encode its exact TLV bytes as Ino1 text without adding the envelope checksum. This extraction is local and offline.

Preserve legacy readers

Wire v0 remains a 12-entry CBOR map with explicit genesis and fork hashes and v0 signing domains. Wire v1 is the 11-element array in this paper. Readers dispatch on container shape and explicit version. They must not guess, silently downgrade or interpret one container as the other.

Re-encoding a decoded v0 address retains its original version and signatures. Issuing a compact replacement requires **fresh recipient authorization and ASP acknowledgment**. Copying v0 signatures into a v1 envelope fails. Existing native Ark addresses and BOLT12 offers retain their original formats.

QR practicality

The mainnet fixture at QR error correction M fits version 18 when uppercase, versus version 20 for the legacy fixture. At four pixels per module plus a four-module border, these render at 388 and 420 pixels respectively. Synthetic decoding passes in both letter cases at M and Q correction levels. Real phone-camera tests remain necessary; size measurements do not prove scanning reliability.

4 Registration and route selection

The recipient wallet obtains an authenticated ASP identity and an Ark pubkey destination. The ASP associates a reusable BOLT12 offer with that destination. The wallet verifies the exact offer bytes and signs the v1 binding. The ASP checks recipient authority, persists the association and countersigns it.

The wallet must persist its registration request and compare the returned binding with that exact request, including revision and validity. A response that merely belongs to the same wallet is not enough. An access token can control registration access; it cannot replace recipient authorization.

Sender	Intended route
Ark wallet using the same ASP	Compare full ASP keys; use the native Ark destination.
Ark wallet using another ASP	Extract the offer; use Lightning with safe delivery preparation.
Lightning-only wallet	Extract the offer; use normal invoice negotiation only when receiver-managed delivery is supported.

No resolver requirement

The address contains both destinations. It does not carry a URL to fetch them or depend on an online recipient directory. A lightweight extractor can recover the BOLT12 offer offline. Normal invoice requests use the offer's embedded Lightning reachability information.

A full ASP identity key is not an HTTP hostname. If an application delivery profile uses a separate ASP API, its endpoint must be configured or authenticated independently. That transport question is distinct from address extraction and is not solved by inventing a resolver dependency.

Freshness and rotation

An embedded offer can expire, be revoked or lose liquidity. Offline verification cannot prove current availability. The receiver must check the active persisted mapping before issuing an invoice or preparing delivery. Payment-status responses, where used, must bind a fresh request nonce, revision, destination and expiry.

Changing the offer requires a newly authorized binding. ASP key rotation requires a newly authenticated identity or an explicitly specified rotation mechanism; a replacement key cannot authorize itself. Remembering revisions helps detect rollback but does not prevent a malicious ASP from equivocating. Do not silently substitute another offer.

5 Payment preparation and recovery

The address establishes which routes the recipient authorized. It does not itself make Lightning settlement atomic with an Ark receive. Safe delivery is a separate implementation requirement, not a property of the checksum or signatures.

Bind one payment intent

Use a unique persisted intent identifier. Bind the destination, revision, exact offer, invoice, payment hash, invoice amount in millisats, recipient net amount in sats, fees, inventory reservation and expiry. Retrying the same intent must return the same result; changing its parameters must fail rather than start another payment.

Show the sender the route, recipient net receipt, maximum total debit and routing fee limit before approval. If fees or available inventory change, request a new approval. Never silently lower the recipient's amount. Invoice and offer validation continue to follow BOLT12.

Prepare delivery before irreversible settlement

The receiving ASP reserves Ark inventory and prepares an enforceable conditional receive for the recipient. The recipient validates and durably stores the recovery material before the settlement condition can be released. The delivery design must specify who controls the preimage, which payment hash is used, timeouts, funding ancestry and claim/refund paths.

The target is recipient-controlled settlement release after safe preparation. This remains a requirement to prove, not a completed construction. A signed acknowledgment or a Lightning preimage alone is not proof that the recipient can recover Ark value if the ASP disappears.

Crash and failure behavior

Persist preparation, reservations and payment state before external actions. Reconcile uncertain outcomes after restart. A timeout is not a definitive failure: do not issue and pay a replacement invoice while the original may have settled. Concurrent requests must not spend the same source VTXO or reserve the same inventory twice.

Cancellation must wait until it is safe to release reservations. Quote expiry must not release inventory backing an in-flight HTLC. Offline recipients need a separately verified delivery design; an ordinary custodial credit is not an acceptable silent fallback.

Liquidity is still required

Lightning transfers channel balance between services. It does not turn channel backing into Ark backing. The receiving ASP still needs Ark inventory and recovery capacity. Quotes must cover nonrecoverable costs and bounded failure/rebalancing costs without assuming an ASP subsidy. Recovery allocations, service fees and routing fees must be shown separately.

6 Evidence and release boundary

As of 4 October 2026, compact v1 exists in the protocol fixture codec and experimental Rust ASP/wallet branches. New authorizations use v1; readers retain v0 verification and encoding. An opt-in experimental CLN plugin also verifies bindings and extracts offers. These are address capabilities, not a deployed Sideflash payment system.

Evidence	What it establishes
Four frozen v0/v1 fixtures	Mainnet/regtest canonical bytes, signatures and exact native/offer extraction agree in the tested codecs.
Rust branch checks	Six focused codec tests per branch, workspace compilation, and the feature-enabled wallet build pass.
Python and QR checks	Signed-field mutations, malformed input, version separation and synthetic QR round-trips pass.
Experimental CLN plugin	Seven local tests include a manifest handshake, payload mutations and mocked local RPC dispatch. No payment is attempted.

The fixtures use known test keys and expired validity intervals. Never fund them. Agreement between these implementations is not independent human review or a security audit. CLN mocked dispatch does not replace live daemon testing.

Before payment release

Complete persistent registration, wallet UI and capability negotiation. Demonstrate same-ASP payments and two separately configured ASPs delivering over Lightning. Prove recovery after disconnection, duplicate requests, double-spend attempts, crashes, lost replies, expired HTLCs and inadequate liquidity. Test ordinary Lightning-only payers with receiver-managed preparation. Test physical QR scanning and keep compatibility with ordinary Ark and Lightning clients.

References and normative details

[Compact v1 specification and network assignments](#)

[Frozen fixtures and codec tests](#)

[Draft Sideflash BOLT proposal - number unassigned](#)

[BOLT12 offer and invoice protocol](#)

[BIP340 signatures](#)

[BIP350 Bech32m](#)

[RFC8949 CBOR](#)

This revision supersedes v0.1's larger proposed bounds, resolver options and unfrozen map design. The experimental wire specification and frozen vectors define exact bytes. No BOLT number, new Lightning feature bit or consensus change is assigned by this paper.